# Airflow 03 Scheduler

Source originale : https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/scheduler.html

↑↓

Navigate

■

Select

Esc

Close

Scheduler

The Airflow scheduler monitors all tasks and Dags, then triggers the
task instances once their dependencies are complete. Behind the scenes,
the scheduler spins up a subprocess, which monitors and stays in sync with all
Dags in the specified Dag directory. Once per minute, by default, the scheduler
collects Dag parsing results and checks whether any active tasks can be triggered.

The Airflow scheduler is designed to run as a persistent service in an
Airflow production environment. To kick it off, all you need to do is
execute the
airflow
scheduler
command. It uses the configuration specified in
airflow.cfg
.

The scheduler uses the configured
Executor
to run tasks that are ready.

To start a scheduler, simply run the command:

airflow
scheduler

Your Dags will start executing once the scheduler is running successfully.

Note

The first Dag Run is created based on the minimum
start_date
for the tasks in your Dag.

Subsequent Dag Runs are created according to your Dag’s

timetable

.

For Dags with a cron or timedelta schedule, scheduler won’t trigger your tasks until the period it covers has ended e.g., A job with

schedule

set as

@daily

runs after the day

has ended. This technique makes sure that whatever data is required for that period is fully available before the Dag is executed.

In the UI, it appears as if Airflow is running your tasks a day

late

Note

If you run a Dag on a

schedule

of one day, the run with data interval starting on

2019-11-21

triggers after

2019-11-21T23:59

.

Let’s Repeat That

, the scheduler runs your job one

schedule

AFTER the start date, at the END of the interval.

You should refer to

Dag Runs

for details on scheduling a Dag.

Note

The scheduler is designed for high throughput. This is an informed design decision to achieve scheduling

tasks as soon as possible. The scheduler checks how many free slots available in a pool and schedule at most that number of tasks instances in one iteration.

This means that task priority will only come into effect when there are more scheduled tasks

waiting than the queue slots. Thus there can be cases where low priority tasks will be scheduled before high priority tasks if they share the same batch.

For more read about that you can reference

this GitHub discussion

.

Running More Than One Scheduler

Airflow supports running more than one scheduler concurrently – both for performance reasons and for resiliency.

Overview

The

HA

scheduler is designed to take advantage of the existing metadata database.

This was primarily done for operational simplicity: every component already has to speak to this DB, and by

not using direct communication or consensus algorithm between schedulers (Raft, Paxos, etc.) nor another

consensus tool (Apache Zookeeper, or Consul for instance) we have kept the “operational surface area” to a

minimum.

The scheduler now uses the serialized Dag representation to make its scheduling decisions and the rough outline of the scheduling loop is:

Check for any Dags needing a new DagRun, and create them

Examine a batch of DagRuns for schedulable TaskInstances or complete DagRuns

Select schedulable TaskInstances, and whilst respecting Pool limits and other concurrency limits, enqueue them for execution

This does, however, place some requirements on the Database.

Database Requirements

The short version is that users of PostgreSQL 12+ or MySQL 8.0+ are all ready to go – you can start running as

many copies of the scheduler as you like – there is no further set up or config options needed. If you are using a different database please read on.

To maintain performance and throughput there is one part of the scheduling loop that does a number of calculations in memory (because having to round-trip to the DB for each TaskInstance would be too slow) so we

need to ensure that only a single scheduler is in this critical section at once - otherwise limits would not be correctly respected. To achieve this we use database row-level locks (using

SELECT

...

FOR

UPDATE

).

This critical section is where TaskInstances go from scheduled state and are enqueued to the executor, whilst

ensuring the various concurrency and pool limits are respected. The critical section is obtained by asking for

a row-level write lock on every row of the Pool table (roughly equivalent to

SELECT

*

FROM

slot_pool

FOR

UPDATE

NOWAIT

but the exact query is slightly different).

The following databases are fully supported and provide an “optimal” experience:

PostgreSQL 12+

MySQL 8.0+

Warning

MariaDB did not implement the

SKIP

LOCKED

or

NOWAIT

SQL clauses until version

10.6.0

.

Without these features, running multiple schedulers is not supported and deadlock errors have been reported. MariaDB

10.6.0 and following may work appropriately with multiple schedulers, but this has not been tested.

Note

Microsoft SQL Server has not been tested with HA.

Fine-tuning your Scheduler performance

What impacts scheduler’s performance

The Scheduler is responsible for continuously scheduling tasks for execution.

In order to fine-tune your scheduler, you need to include a number of factors:

The kind of deployment you have

how much memory you have available

how much CPU you have available

how much networking throughput you have available

The logic and definition of your Dag structure:

how many Dags you have

how complex they are (i.e. how many tasks and dependencies they have)

The scheduler configuration

How many schedulers you have

How many task instances scheduler processes in one loop

How many new Dag runs should be created/scheduled per loop

How often the scheduler should perform cleanup and check for orphaned tasks/adopting them

In order to perform fine-tuning, it's good to understand how Scheduler works under-the-hood.

You can take a look at the Airflow Summit 2021 talk

Deep Dive into the Airflow Scheduler talk

to perform the fine-tuning.

How to approach Scheduler's fine-tuning

Airflow gives you a lot of "knobs" to turn to fine tune the performance but it's a separate task, depending on your particular deployment, your Dag structure, hardware availability and expectations, to decide which knobs to turn to get best effect for you. Part of the job when managing the deployment is to decide what you are going to optimize for.

Airflow gives you the flexibility to decide, but you should find out what aspect of performance is most important for you and decide which knobs you want to turn in which direction.

Generally for fine-tuning, your approach should be the same as for any performance improvement and optimizations (we will not recommend any specific tools - just use the tools that you usually use to observe and monitor your systems):

it's extremely important to monitor your system with the right set of tools that you usually use to monitor your system. This document does not go into details of particular metrics and tools that you can use, it just describes what kind of resources you should monitor, but you should follow your best practices for monitoring to grab the right data.

decide which aspect of performance is most important for you (what you want to improve)

observe your system to see where your bottlenecks are: CPU, memory, I/O are the usual limiting factors

based on your expectations and observations - decide what is your next improvement and go back to the observation of your performance, bottlenecks. Performance improvement is an iterative process.

What resources might limit Scheduler's performance

There are several areas of resource usage that you should pay attention to:

Database connections and Database usage might become a problem as you want to increase performance and

process more things in parallel. Airflow is known for being "database-connection hungry" - the more Dags

you have and the more you want to process in parallel, the more database connections will be opened. This is generally not a problem for MySQL as its model of handling connections is thread-based, but this might be a problem for Postgres, where connection handling is process-based. It is a general consensus that if you have even medium size Postgres-based Airflow installation, the best solution is to use PGBouncer as a proxy to your database. The Helm Chart for Apache Airflow supports PGBouncer out-of-the-box.

The Airflow Scheduler scales almost linearly with several instances, so you can also add more Schedulers if your Scheduler's performance is CPU-bound.

Make sure when you look at memory usage, pay attention to the kind of memory you are observing. Usually you should look at working memory (names might vary depending on your deployment) rather than total memory used.

What can you do, to improve Scheduler's performance

When you know what your resource usage is, the improvements that you can consider might be:

improve utilization of your resources. This is when you have a free capacity in your system that seems underutilized (again CPU, memory I/O, networking are the prime candidates) - you can take actions like increasing number of schedulers or decreasing intervals for more frequent actions might bring improvements in performance at the expense of higher utilization of those.

increase hardware capacity (for example if you see that CPU is limiting you).

Often the problem with scheduler performance is simply because your system is not "capable" enough and this might be the only way. For example if you see that you are using all CPU you have on machine, you might want to add another scheduler on a new machine - in most cases, when you add 2nd or 3rd scheduler, the capacity of scheduling grows linearly (unless the shared database or similar is a bottleneck).

experiment with different values for the "scheduler tunables". Often you might get better effects by simply exchanging one performance aspect for another. Usually performance tuning is the art of balancing different aspects.

Scheduler Configuration options

The following config settings can be used to control aspects of the Scheduler.

However, you can also look at other non-performance-related scheduler configuration parameters available at

Configuration Reference

in the

[scheduler]

section.

max_dagruns_to_create_per_loop

This changes the number of Dags that are locked by each scheduler when

creating Dag runs. One possible reason for setting this lower is if you

have huge Dags (in the order of 10k+ tasks per Dag) and are running multiple schedulers, you won't want one

scheduler to do all the work.

max_dagruns_per_loop_to_schedule

How many DagRuns should a scheduler examine (and lock) when scheduling

and queuing tasks. Increasing this limit will allow more throughput for

smaller Dags but will likely slow down throughput for larger (>500

tasks for example) Dags. Setting this too high when using multiple

schedulers could also lead to one scheduler taking all the Dag runs

leaving no work for the others.

use_row_level_locking

Should the scheduler issue

SELECT

...

FOR

UPDATE

in relevant queries.

If this is set to False then you should not run more than a single

scheduler at once.

pool_metrics_interval

How often (in seconds) should pool usage stats be sent to StatsD (if

statsd_on is enabled). This is a

relatively

expensive query to compute

this, so this should be set to match the same period as your StatsD roll-up

period.

ti_metrics_interval

How often (in seconds) should task instance (scheduled, queued, running and deferred) stats be sent to StatsD

(if statsd_on is enabled). This is a

relatively

expensive query to compute

this, so this should be set to match the same period as your StatsD roll-up

period.

orphaned_tasks_check_interval

How often (in seconds) should the scheduler check for orphaned tasks or dead

SchedulerJobs.

This setting controls how a dead scheduler will be noticed and the tasks it

was “supervising” get picked up by another scheduler. The tasks will stay

running, so there is no harm in not detecting this for a while.

When a SchedulerJob is detected as “dead” (as determined by

scheduler_health_check_threshold

) any running or

queued tasks that were launched by the dead process will be “adopted” and

monitored by this scheduler instead.

max_tis_per_query

The batch size of queries in the scheduling main loop. This should not be greater than

core.parallelism

. If this is too high then SQL query performance may be impacted by

complexity of query predicate, and/or excessive locking.

Additionally, you may hit the maximum allowable query length for your db.

Set this to 0 to use the value of

core.parallelism

.

scheduler_idle_sleep_time

Controls how long the scheduler will sleep between loops, but if there was nothing to do

in the loop. i.e. if it scheduled something then it will start the next loop

iteration straight away. This parameter is badly named (historical reasons) and it will be

renamed in the future with deprecation of the current name.

Was this entry helpful?